Role

Act as a Principal Software Architect and Expert in Agentic AI Systems, the Model Context Protocol (MCP), and highly concurrent backend engineering (Go/Rust).

Context

I want to build the "holy grail of agentic development": a highly powerful, concurrent MCP server called the "Concurrent Workspace & Swarm Orchestrator" (CWSO). Currently, MCP servers are mostly synchronous and single-lane. CWSO will act as a parallel, virtualized project manager for an LLM (like Gemini via CLI) and its specialized sub-agents.

Key features and concepts from our design discussion include:

- **Shadow Workspaces:** Ephemeral, containerized Git branches that allow multiple sub-agents to edit the same codebase concurrently without relying on restrictive file-locking.
- **Semantic AST Queries (query_ast):** Fast, concurrent parsing of Abstract Syntax Trees across languages instead of basic text-based grep or read_file.
- **Asynchronous Tool Calling & Polling:** A "fire and forget" dispatching system (dispatch_concurrent_jobs) that returns a job ID immediately.
- **Real-Time Resource Streaming:** Using Server-Sent Events (SSE) via MCP Resources to stream the live status of background threads.
- **Conflict-Free Multi-Agent Merging (merge_concurrent_results):** Server-side Git-level logic to auto-resolve parallel edits or format merge conflicts for the main LLM to resolve intelligently.
- **Deterministic Orchestration:** Shifting coordination logic out of the LLM into a rigid Go/Rust server, contrasting with model-driven coordination (like the leaked Claude Code architecture).

Task

Based on the ideas above, create a comprehensive, technical architectural blueprint and a step-by-step development plan to build the CWSO application from scratch.

Structure the Plan as follows:

1. **System Architecture:** Define the tech stack (e.g., Go vs. Rust), container orchestration method, Git tree manipulation layer, and the MCP JSON-RPC interface.
2. **Core Workflows:** Step-by-step flowcharts or descriptions for the "Fire-and-Forget

Dispatch", "Background Processing", and the "Merge & Resolution" loop.
3. **API & Tool Definitions:** Draft the specific MCP tool schemas (inputs/outputs) for query_ast, create_shadow_workspace, dispatch_concurrent_jobs, and merge_concurrent_results.
4. **Development Roadmap:** Break the project down into logical, achievable milestones (Phase 1: Minimal Viable Synchronous Server -> Phase 4: Full Containerized Swarm).
5. **Technical Bottlenecks:** Identify the biggest risks (e.g., multi-language AST parsing overhead, container startup latency) and propose mitigation strategies.